Java Exercise 1:
Bank Account with OOP Concepts

Duration: 30 minutes

Instructions: You are tasked with implementing a bank account system implementing inheritance, encapsulation, and polymorphism. The solution should be modular, scalable, and robust, providing flexibility for extending the system to handle different types of bank accounts in the future. Accomplish the exercise based on the details listed below.

Requirements:

- Define a parent class that allows you to deposit, withdraw, and get balance. In addition, the account can be frozen and unfrozen. Deposit should only work if the account is not frozen. Withdraw should not exceed the balance.
- Define a child class that will allow you to specifically initialize the owner of the account using its constructor.
- Define a main class that demonstrates the polymorphism (upcasting) where functions implemented on the parent class are invoked and performed.
- For amounts or balances implemented on the program, only retain two decimal places. *Hint:* You may use `printf`.
- All of the classes must have Java Documentation and are checked with Check Style.
- Before submission, your program should be tested thoroughly and is functional. You may refer to the next page for the test cases.

Class Design:

- Parent Class: **BankAccount**
  - **Fields**:
    - `double balance`
    - `boolean isFrozen`
  - **Constructor:**
    - `BankAccount()`: Initializes `balance` with 0, `isFrozen` with false.
  - **Methods**:
    - `deposit(double amount)`: The function deposits an `amount`. It checks if the account is not frozen and the `amount` deposited is valid. Proper error or invalid message must be displayed when doing the transaction.
    - `withdraw(double amount)`: The function withdraws an amount. It checks if the account is not frozen and the `amount` to be withdrawn is valid. Proper error or invalid message must be displayed when doing the transaction.
    - `getBalance()`: Returns the current balance.
    - `freezeAccount`(): Sets account to frozen.
    - `unfreezeAccount`(): Sets account to unfrozen.
    - `isFrozen()`: Returns the status of an account if it is frozen.

- Child Class: **SavingsAccount**
  - **Fields**:
    - String `ownerName`: Holds the name of the owner's account.
  - **Constructor**:
    - `SavingsAccount`(`String ownerName`): Initializes the account owner's name .
  - **Methods**:
    - `getOwnerName`(): returns the `ownerName`.

Test Case	Description	Operations	Expected Output
1	Create a savings account	BankAccount acc1 = new SavingsAccount("Test0")	Account Owner: Test0.
2	Deposit with valid amount	Call deposit(1000.125).	Amount deposited: 1000.13 Current balance: 1000.13
3	Deposit with zero amount	Call deposit(0).	Amount deposited: 0.0 Invalid amount. Must be greater than 0
4	Deposit with negative amount	Call deposit(-500).	Amount deposited: -500.0 Invalid amount. Must be greater than 0
5	Withdraw with sufficient funds	Call withdraw(500)	Amount withdrawn: 500.0 Current balance: 500.13
6	Withdraw with insufficient funds	Call withdraw(1500).	Amount withdrawn: 1500.0 Insufficient balance. Current balance: 500.13
7	Withdraw with negative amount	Call withdraw(-100).	Amount withdrawn: -100.0 Invalid amount. Must be greater than 0
8	Deposit when account is frozen	1. Call freezeAccount(). 2. Call deposit(11500).	Account is FROZEN. Unable to transact.
9	Withdraw when account is frozen	Call withdraw(500).	Account is FROZEN. Unable to transact.
10	Unfreeze account and withdraw	1. Call unfreezeAccount(). 2. Call withdraw(100).	Amount withdrawn: 100.0 Current balance: 400.13

Criteria	5	4	3	2	1	0
Correctness	*Correctly solves the problem for all test cases.*	*Solves most cases, with minor errors in edge cases.*	*Works for some test cases, but has issues.*	*Incorrect for most test cases.*	*Only correct in specific cases or completely incorrect.*	*Doesn't compile or run.*
Algorithmic Design and Data Handling	-	-	-	*The program handles data efficiently, avoiding unnecessary steps and working well with large inputs.*	*Data handling is okay but could be more efficient with fewer extra steps.*	*Data is handled poorly, making the program slow or incorrect.*
Code Readability and Maintainability	-	-	-	*The code is clean, well-organized, and easy to follow. It features proper use of comments and meaningful variable names. Additionally, there are no violations from Checkstyle.*	*The code is understandable but could be better organized or clearer (e.g., some variable names are unclear or there is a lack of comments). It has minimal or minor violations from Checkstyle, such as improper indentation or spacing.*	*The code is messy, disorganized, or hard to understand, with major Checkstyle violations.*
Timeliness	-	-	-	-	*Submitted the exercise on or before the deadline.*	*Submitted the exercise after the deadline.*
Total	10					
Passing Score	7					